Learning Objectives

Learners will be able to...

- **Define a priority queue**
- **Differentiate a priority queue from a traditional queue**
- **Implement a priority queue using an array, vector, and linked list**

Priority Queue

What is a Priority Queue?

A **priority queue** is an abstract data type (ADT) similar to a regular queue or stack but with an added dimension: **priority**. Unlike standard queues where the first in, first out (FIFO) principle applies, elements in a priority queue are dequeued based on their priority; higher priority elements are removed first. This data structure is particularly useful in algorithms and systems that require a more flexible and nuanced approach to data processing, such as task scheduling, data compression algorithms, and network traffic management.

The image on the left shows how a priority queue works. Elements in the priority queue have a priority indicator. Those with the highest priority (red elements) are moved to the front of the queue. Then come elements with a decreasing priority value (yellow elements). When adding a new element (green), you then have to calculate its position in the priority queue.

Priorities can be represented in a variety of ways. For the purposes of this assignment, we are going to keep our priority simple. Our priority queue is going to work with integers. The larger the integer, the greater the priority. Essentially, our examples will store values in descending order.

Basic Operations

Here are the main operations associated with priority queues:

- **Enqueue**: Insert an element to the queue with an associated priority.
- **Dequeue**: Delete the highest-priority element from the queue. If two elements have the same priority, the queue may resort to FIFO.
- **Peek**: Views the highest-priority element without removing it from the queue.
- **IsEmpty**: Checks if the priority queue is empty.
- **Size**: Returns the number of elements in the priority queue.

Implementations

Priority Queues can be implemented using various underlying data structures, the most common of which are:

- **Arrays or Linked Lists**: Simpler but generally less efficient, $O(n)$ time complexity for insertion.
- **Binary Heaps**: More complex but provide better efficiency, offering

$O(logn)$ time complexity for both insertion and removal.
- **Fibonacci Heaps**: Advanced data structures that offer better amortized time complexities.

In this assignment, we will explore different implementations of priority queues, starting with the most straightforward array-based approach, we will touch on heaps in a later assignment.

Array-Based Implementation

Representing a Priority Queue with an Array

In its most straightforward form, a priority queue can be implemented using a simple array. Although this approach is less efficient than using more advanced data structures like heaps, it serves as a good introduction to the concept. In this section, we'll explore how to build a priority queue using an array as the underlying data structure.

Key Operations and Their Time Complexities

Here are the key operations and their respective time complexities for an array-based Priority Queue:

- **Enqueue**: $O(n)$
- **Dequeue**: $O(n)$
- **Peek**: $O(1)$
- **IsEmpty**: $O(1)$
- **Size**: $O(1)$

Note that both the `Enqueue` and `Dequeue` operations have a time complexity of $O(n)$ because in the worst-case scenario we may need to traverse the entire array to find the correct position for insertion or deletion.

Implementation

Create the `ArrayPriorityQueue` class which has three attributes: `capacity`, `array`, and `size`. `capacity` refers to the number of elements the queue can hold, `array` is the underlying data structure, and `size` refers to how many elements are currently in the priority queue. The constructor takes a value for the capacity and initializes the three attributes.

```
class ArrayPriorityQueue {
private:
    int capacity;
    int* array;
    int size;

public:
    // Constructor
    ArrayPriorityQueue(int capacity) {
        this->capacity = capacity;
        array = new int[this->capacity];
        size = 0;
    }

    ~ArrayPriorityQueue() {
        delete[] array;
    }
};
```

When enqueueing an element, let's start by checking for two special cases. If the queue is full, throw an exception with a message to the user. If the queue is empty, set the element at index `size` to the given value. Then increment `size` by `1` and use return to exit the method.

```
void enqueue(int value) {
    if (isFull()) {
        throw std::overflow_error("Priority queue is full");
    }

    if (isEmpty()) {
        array[size++] = value;
        return;
    }
}
```

If we need to add an element and these two special cases do not apply, we need to place the element in its proper position. The larger the value, the greater the priority. Start by creating the variable `i`. Use `i` in a for loop that iterates backwards over the array. If the given value is less than the value at index `i`, shift the value at index `i` to the right. If the given value is greater than or equal to the value at index `i`, break out of the loop. Then set the element at `i + 1` to the given value and increment `size` by `1`.

```
void enqueue(int value) {
    if (isFull()) {
        throw std::overflow_error("Priority queue is full");
    }

    if (isEmpty()) {
        array[size++] = value;
        return;
    }

    int i;
    for (i = size - 1; i >= 0; i--) {
        if (value < array[i]) {
            array[i + 1] = array[i];
        } else {
            break;
        }
    }
    array[i + 1] = value;
    size++;
}
```

When dequeueing an element, first check to see if the queue is empty. If so, throw an exception with a message to the user. If there are elements in the queue, decrement the value `size` by `1` and then return the element at this new index.

```
int dequeue() {
    if (isEmpty()) {
        throw std::underflow_error("Priority queue is empty");
    }

    return array[--size];
}
```

To peek at the highest priority element, return the value stored at `size - 1`. However, do not alter the value of `size`.

```
int peek() {
    if (isEmpty()) {
        throw std::underflow_error("Priority queue is empty");
    }

    return array[size - 1];
}
```

Checking the size of the priority queue is just an accessor method that returns the `size` attribute.

```
  // Check the size of the Priority Queue
  int getSize() const {
        return size;
  }
```

Lastly, a priority queue is empty when `size` is equal to `0`. Have the `isEmpty` method return the result from this boolean expression.

```
bool isEmpty() const {
    return size == 0;
}

bool isFull() const {
    return size == capacity;
}
```

Create the `main` method which instantiates an array-based priority queue with a capacity of `5`. Add the values `3`, `2`, and `4`. Use `peek` to display the value with the highest priority, print the size, dequeue the element with the highest priority, and print the size once again.

```
int main() {
    ArrayPriorityQueue myPQ(5);
    myPQ.enqueue(3);
    myPQ.enqueue(2);
    myPQ.enqueue(4);
    std::cout << "Highest priority element: " << myPQ.peek() <<
        std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    std::cout << "Dequeue operation result: " << myPQ.dequeue()
        << std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;

    return 0;
}
```

You should see the following output:

```
Highest priority element: 4
Priority queue size: 3
Dequeue operation result: 4
Priority queue size: 2
```

▼ Code

Your code should look something like this:

```
class ArrayPriorityQueue {
private:
```

```
    int capacity;
    int* array;
    int size;

public:
    // Constructor
    ArrayPriorityQueue(int capacity) {
        this->capacity = capacity;
        array = new int[this->capacity];
        size = 0;
    }

    ~ArrayPriorityQueue() {
        delete[] array;
    }

    // Enqueue operation with priority handling
    void enqueue(int value) {
        if (isFull()) {
            throw std::overflow_error("Priority queue is full");
        }

        if (isEmpty()) {
            array[size++] = value;
            return;
        }

        int i;
        for (i = size - 1; i >= 0; i--) {
            if (value < array[i]) {
                array[i + 1] = array[i];
            } else {
                break;
            }
        }
        array[i + 1] = value;
        size++;
    }

    // Dequeue operation
    int dequeue() {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }

        return array[--size];
    }

    // Peek operation
    int peek() {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }

        return array[size - 1];
    }
```

```
    // Check the size of the Priority Queue
    int getSize() const {
        return size;
    }

    // Check if the Priority Queue is empty
    bool isEmpty() const {
        return size == 0;
    }

    // Check if the priority queue is full
    bool isFull() const {
        return size == capacity;
    }
};

int main() {
    ArrayPriorityQueue myPQ(5);
    myPQ.enqueue(3);
    myPQ.enqueue(2);
    myPQ.enqueue(4);
    std::cout << "Highest priority element: " << myPQ.peek() <<
        std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    std::cout << "Dequeue operation result: " << myPQ.dequeue()
        << std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;

    return 0;
}
```

challenge

Try this variation

Update the enqueue method so that the conditional checks to see if the given value is greater than the element at index `i`. Then run the program again.

```
if (value > array[i]) {
    array[i + 1] = array[i];
} else {
```

▼ **What is happening?**

If we change the boolean operator, we are now shifting elements in the array when the given value is greater than the current element. That is, smaller values get moved to the front of the queue. The queue goes from being arranged from largest to smallest, to now being smallest to largest. Elements with a smaller value are given priority over elements with a larger value.

In the next section, we'll see how to implement a priority queue with a linked list.

Linked List-Based Implementation

Representing a Priority Queue with a Linked List

Another straightforward way to implement a priority queue is by using a Linked List as the underlying data structure. This method may offer some advantages over the array-based approach, particularly in terms of dynamic sizing. In this section, we will delve into the specifics of a Linked List-based Priority Queue.

Key Operations and Their Time Complexity

Here are the key operations and their respective time complexities for a Linked List-based Priority Queue:

- **Enqueue**: $O(n)$
- **Dequeue**: $O(1)$
- **Peek**: $O(1)$
- **IsEmpty**: $O(1)$
- **Size**: $O(1)$

As with the array-based approach, the `enqueue` operation may require traversing the entire list to find the correct position for the new item, leading to a time complexity of $O(n)$. However, `dequeue` and `peek` are more efficient with $O(1)$ since we maintain a reference to the head of the list.

Implementation

Since we are using a linked list, we need a `Node` class. This is going to be a singly linked list, so our node has a `next` pointer and a `data` attribute to store information.

```
class Node {
public:
    int data;
    Node* next;

    Node(int data) : data(data), next(nullptr) {}
};
```

Now we can create the `LinkedListPriorityQueue` class. It has two attributes, head (keeps track of the first node) and `size` (keeps track of the `size` of the list). In the constructor, set `head` to `nullptr` and `size` to `0`.

```
class LinkedListPriorityQueue {
private:
    Node* head;
    int size;

public:
    // Constructor
    LinkedListPriorityQueue() : head(nullptr), size(0) {}

    ~LinkedListPriorityQueue() {
        while (!isEmpty()) {
            dequeue();
        }
    }

};
```

When we enqueue an element, we either put the element directly at the head or iterate through the list to find the correct location. A node can go to the head if the list is empty or the given value is greater than the value already stored at the head.

```
    // Enqueue operation with priority handling
    void enqueue(int value) {
        Node* newNode = new Node(value);
        if (isEmpty() || value > head->data) {
            newNode->next = head;
            head = newNode;
        }

    }
```

Otherwise, we need to iterate over the linked list until either we reach the end or the value of the next node is less than or equal to the value of the new node. Be sure to properly manage the pointers. The current node should point to the new node, and the new node should point to the node after the current node. Then increment `size` by `1`.

```
    // Enqueue operation with priority handling
    void enqueue(int value) {
        Node* newNode = new Node(value);
        if (isEmpty() || value > head->data) {
            newNode->next = head;
            head = newNode;
        } else {
            Node* current = head;
            while (current->next != nullptr && current->next-
        >data <= value) {
                current = current->next;
            }
            newNode->next = current->next;
            current->next = newNode;
        }
        size++;
    }
```

To dequeue an element, start by asking if the list is empty. If yes, throw an exception with a message for the user. Then create a temporary variable that stores the value in the head node. Change the new head to the next node in the list. After that, decrement `size` by `1`. Finally, return the temporary variable.

```
    // Dequeue operation
    int dequeue() {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        int value = head->data;
        Node* temp = head;
        head = head->next;
        delete temp;
        size--;
        return value;
    }
```

When peeking at the element with the highest priority, return the data found at the head of the list. Do not change any pointers.

```
    // Peek operation
    int peek() const {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        return head->data;
    }
```

Checking the size of the priority queue is just an accessor method that returns the `size` attribute.

```
    // Check the size of the Priority Queue
    int getSize() const {
        return size;
    }
```

Lastly, we need the `isEmpty` helper method. A linked list is empty when the head of the list is `nullptr`.

```
    // Check if the Priority Queue is empty
    bool isEmpty() const {
        return head == nullptr;
    }
```

Create the `main` method which instantiates a linked list-based priority queue. Add the values `3`, `2`, and `4`. Use `peek` to display the value with the highest priority, print the size, dequeue the element with the highest priority, and print the size once again.

```
int main() {
    LinkedListPriorityQueue myPQ;
    myPQ.enqueue(3);
    myPQ.enqueue(2);
    myPQ.enqueue(4);
    std::cout << "Highest priority element: " << myPQ.peek() <<
        std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    std::cout << "Dequeue operation result: " << myPQ.dequeue()
        << std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    return 0;
}
```

You should see the following output:

```
Highest priority element: 4
Priority queue size: 3
Dequeue operation result: 4
Priority queue size: 2
```

▼ Code

Your code should look something like this:

```
#include <iostream>
#include <stdexcept>

class Node {
public:
    int data;
```

```
    Node* next;

    Node(int data) : data(data), next(nullptr) {}
};

class LinkedListPriorityQueue {
private:
    Node* head;
    int size;

public:
    // Constructor
    LinkedListPriorityQueue() : head(nullptr), size(0) {}

    ~LinkedListPriorityQueue() {
        while (!isEmpty()) {
            dequeue();
        }
    }

    // Enqueue operation with priority handling
    void enqueue(int value) {
        Node* newNode = new Node(value);
        if (isEmpty() || value > head->data) {
            newNode->next = head;
            head = newNode;
        } else {
            Node* current = head;
            while (current->next != nullptr && current->next-
        >data <= value) {
                current = current->next;
            }
            newNode->next = current->next;
            current->next = newNode;
        }
        size++;
    }

    // Dequeue operation
    int dequeue() {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        int value = head->data;
        Node* temp = head;
        head = head->next;
        delete temp;
        size--;
        return value;
    }

    // Peek operation
    int peek() const {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
```

```
        return head->data;
    }

    // Check the size of the Priority Queue
    int getSize() const {
        return size;
    }

    // Check if the Priority Queue is empty
    bool isEmpty() const {
        return head == nullptr;
    }
};

int main() {
    LinkedListPriorityQueue myPQ;
    myPQ.enqueue(3);
    myPQ.enqueue(2);
    myPQ.enqueue(4);
    std::cout << "Highest priority element: " << myPQ.peek() <<
        std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    std::cout << "Dequeue operation result: " << myPQ.dequeue()
        << std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    return 0;
}
```

Vector-Based Implementation

Representing a Priority Queue with a Vector

Vectors in C++ are dynamic arrays that resize themselves automatically when elements are added or removed. They provide the benefits of arrays while removing the need for manual size management. In this section, we explore implementing a Priority Queue using a `std::vector`.

Key Operations and Their Time Complexities

Here are the key operations and their respective time complexities for a vector-based Priority Queue:

- **Enqueue**: $O(n)$
- **Dequeue**: $O(1)$
- **Peek**: $O(1)$
- **IsEmpty**: $O(1)$
- **Size**: $O(1)$

The time complexity of the `enqueue` operation is $O(n)$ because, in the worst-case scenario, you may need to traverse the entire list to find the correct position for the new item. On the other hand, `dequeue` and `peek` operations are generally $O(1)$ if we keep track of the front and rear of the queue.

Implementation

Start by including the `vector` header. Then create the `VectorPriorityQueue` class, which only has a vector as an attribute. This is where all of the data for the priority queue will be stored. Next, instantiate the vector in the constructor.

```
class VectorPriorityQueue {
private:
    std::vector<int> vector;

public:
    // Constructor
    VectorPriorityQueue() {}

};
```

Since we are working with a vector, we are going to use methods associated with this class for our priority queue operations. Start by creating the `enqueue` method that takes an integer value. We also need the variable `indexToInsert` which will be the index to use when inserting the given value. Iterate over the vector with a for loop. Create the `current` variable and set its value to the element in the loop. If the given value is greater than the `current` value, set `indexToInsert` to the loop variable and break out of the loop. If the value is not greater than current, increment `indexToInsert`. After the loop ends, use the `insert` method and `indexToInsert` to insert the given value at the `correct` position.

```
    // Enqueue operation with priority handling
    void enqueue(int value) {
        int indexToInsert = 0;
        // Find the correct index to insert the item based on
        priority
        for (int i = 0; i < vector.size(); i++) {
            int current = vector[i];
            // Compare the new item with the current item for
        priority
            if (value > current) {
                indexToInsert = i;
                break;
            }
            // If the new item has higher priority, increment
        the index
            indexToInsert++;
        }
        // Insert the item at the determined index
        vector.insert(vector.begin() + indexToInsert, value);
    }
```

When dequeueing an element, first check to see if the priority queue is empty. If so, throw an exception with a message to the user. Otherwise, use the `erase` method to remove the first element in the priority queue.

```
    // Dequeue operation
    int dequeue() {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        int value = vector[0];
        vector.erase(vector.begin());
        return value;
    }
```

To peek at the highest priority element, first check to see if the priority queue is empty. If yes, throw an exception with a message to the user. Otherwise, return the first element in the priority queue.

```
    // Peek operation
    int peek() const {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        return vector[0];
    }
```

Checking the size of the priority queue is done with the `vector` method `size`. This will return the length of the `vector` attribute (the priority queue).

```
    // Check the size of the Priority Queue
    int getSize() const {
        return vector.size();
    }
```

Lastly, create the `isEmpty` helper method. Vectors have a method named `empty` which is what we will call on the priority queue. This method will return a boolean value.

```
    // Check if the Priority Queue is empty
    bool isEmpty() const {
        return vector.empty();
    }
```

Create the `main` function which instantiates a vector-based priority queue. Add the values 3, 2, and 4. Use peek to display the value with the highest priority, print the `size`, dequeue the element with the highest priority, and print the `size` once again.

```
int main() {
    VectorPriorityQueue myPQ;
    myPQ.enqueue(3);
    myPQ.enqueue(2);
    myPQ.enqueue(4);
    std::cout << "Highest priority element: " << myPQ.peek() <<
        std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    std::cout << "Dequeue operation result: " << myPQ.dequeue()
        << std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    return 0;
}
```

You should see the following output:

```
Highest priority element: 4
Priority queue size: 3
Dequeue operation result: 4
Priority queue size: 2
```

▼ Code

Your code should look something like this:

```cpp
#include <iostream>
#include <vector>
#include <stdexcept>

class VectorPriorityQueue {
private:
    std::vector<int> vector;

public:
    // Constructor
    VectorPriorityQueue() {}

    // Enqueue operation with priority handling
    void enqueue(int value) {
        int indexToInsert = 0;
        // Find the correct index to insert the item based on
        priority
        for (int i = 0; i < vector.size(); i++) {
            int current = vector[i];
            // Compare the new item with the current item for
        priority
            if (value > current) {
                indexToInsert = i;
                break;
            }
            // If the new item has higher priority, increment
        the index
            indexToInsert++;
        }
        // Insert the item at the determined index
        vector.insert(vector.begin() + indexToInsert, value);
    }

    // Dequeue operation
    int dequeue() {
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        int value = vector[0];
        vector.erase(vector.begin());
        return value;
    }

    // Peek operation
    int peek() const {
```

```
        if (isEmpty()) {
            throw std::underflow_error("Priority queue is
        empty");
        }
        return vector[0];
    }

    // Check the size of the Priority Queue
    int getSize() const {
        return vector.size();
    }

    // Check if the Priority Queue is empty
    bool isEmpty() const {
        return vector.empty();
    }
};

int main() {
    VectorPriorityQueue myPQ;
    myPQ.enqueue(3);
    myPQ.enqueue(2);
    myPQ.enqueue(4);
    std::cout << "Highest priority element: " << myPQ.peek() <<
        std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    std::cout << "Dequeue operation result: " << myPQ.dequeue()
        << std::endl;
    std::cout << "Priority queue size: " << myPQ.getSize() <<
        std::endl;
    return 0;
}
```

In the following section, we will compare the performance of different implementations of priority queues to help you understand the trade-offs involved.

Analyzing Priority Queue Implementations

Comparing Different Implementations of Priority Queue

We have so far looked at implementing a priority queue with array-based, linked list-based, and vector-based implementations individually. Let's now compare the array-based priority queue with other common implementations: linked list and dynamic array.

Implementation Type	Enqueue	Dequeue	Peek	IsEmpty	Space Complexity
Array-Based	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$
Dynamic Array	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(1)$ (amortized)

▼ What does "amortized" mean?
The term "amortized" is used to describe an average time per operation over a sequence. In the context of a dynamic array, although individual operations can be expensive due to resizing, when averaged over a large number of operations, the cost becomes constant.

Array-Based Priority Queue

Advantages:

- Simple to implement
- Constant time complexity for `peek` and `isEmpty` operations

Disadvantages:

- Fixed size limits the number of elements that can be stored
- Both `enqueue` and `dequeue` operations are $O(n)$, making it inefficient for large datasets

Linked List-Based Priority Queue

Advantages:

- Dynamic size
- Constant time complexity for `peek` and `isEmpty` operations

Disadvantages:

- $O(n)$ time complexity for `enqueue` and `dequeue` operations
- Extra memory overhead due to the pointers

Dynamic Array-Based Priority Queue

Advantages:

- Dynamic size, can grow as needed
- Constant time complexity for `peek` and `isEmpty` operations

Disadvantages:

- $O(n)$ time complexity for `enqueue` and `dequeue` operations
- Amortized constant space complexity

Formative Assessment 1

Formative Assessment 2